CONTOSO RETAIL DW

End-to-End BI Analytics Project

Document 1 of 3 — Database Architecture

Sections 0–9: Project Overview, SQL Architecture, [gen] Extension, View Layer, Star Schema & Python Export Pipeline

Field	Detail
Author	Waleed Mouhammed
Programme	DEPI — Data Analysis Track
Engine	SQL Server 2025 (T-SQL)
Database	ContosoRetailDW
Document	1 of 3 — Database Architecture & SQL Layer
Version	1.0 — March 2026
AI Collaboration	Anthropic Claude — Architectural co-designer

Executive Summary

The Contoso Retail DW Analytical Extension Project transforms Microsoft's standard Contoso Retail Data Warehouse — a rich but analytically incomplete sample database — into a production-grade, executive-ready business intelligence platform. The source database covers online sales, physical store sales, inventory, promotions, products, stores, and employees. However, it has six structural blind spots that prevent executive-grade analysis: no customer acquisition data, no payment method data, no fulfilment lifecycle data, no marketing spend data, no customer satisfaction data, and no event-level return data.

This project closes every one of those gaps through a carefully engineered synthetic data extension built on a strict three-layer SQL architecture, then delivers a Power BI platform serving six executive personas — CEO, CFO, COO, CSO, CMO, and Product Manager — across 89 documented business questions.

Three interlocking architectural pillars define the project:

- A three-schema SQL architecture ([dbo] source → [gen] synthetic extension → [dim]/[fact] semantic views) that extends the dataset without corrupting the source and enforces the principle that all business logic is pushed as far upstream into SQL as possible — what this document calls Roche's Maxim.
- A Python-driven Parquet export pipeline bridging the SQL semantic layer to a two-file Power BI model, engineered to handle 31.5 million rows reliably across local development and cloud production environments.
- Seven DAX analytical pipelines comprising 49 measures, covering revenue economics, customer lifetime value, marketing ROI, fulfilment efficiency, returns analytics, product performance, and customer satisfaction.

Current analytical coverage: 79 of 89 business questions fully answerable (89%). Document 2 of this series defines the engineering path to 100% coverage.

1. Project Overview & Objectives

1.1 Business Context & the Gap Problem

Contoso Retail is Microsoft's canonical reference retailer — a fictional multinational consumer electronics company whose Data Warehouse is the industry standard for demonstrating BI tooling. While the source database is rich in transactional history, it was designed as a demonstration dataset for basic reporting, not executive decision support. Six structural gaps prevent it from serving a modern retail analytics agenda:

Gap	Missing Data Domain	Affected Personas	Business Impact
G1	Customer acquisition channel — HOW customers first discovered the brand	CMO, CFO, CEO	CAC by channel, ROAS, paid/organic mix, CLV by acquisition source — all impossible without this data
G2	Payment method per order — HOW customers paid for transactions	CFO, COO	Digital payment adoption trend, payment mix analysis, checkout friction metrics — all impossible
G3	Order fulfilment lifecycle — shipping dates, delivery dates, SLA compliance	COO, CEO, CFO	On-time delivery rate, avg lead time, SLA attainment, warehouse vs carrier delay attribution — all impossible
G4	Marketing spend by channel and period — investment amounts	CMO, CFO, CEO	ROAS, CPC, CPA, budget allocation optimisation, acquisition payback period — all impossible
G5	Customer satisfaction — NPS and CSAT survey responses	CEO, CSO, CMO, COO	Net Promoter Score, promoter/detractor analysis, CSAT trends, CLV-NPS correlation — all impossible
G6	Event-level return data — when products were returned and why	COO, CFO, PM	Return reason breakdown, return lag analysis, operational vs preference return split — all impossible

1.2 Executive Personas & Analytical Domains

All analytical deliverables are organised around six executive personas. Each persona represents a distinct decision-making domain with its own business questions, required KPIs, and dedicated Power BI dashboard page.

Persona	Title	Core Decision Domain	Primary KPIs
CEO	Chief Executive Officer	Enterprise performance, growth, strategic direction	Total Revenue, YoY Growth %, Gross Margin %, NPS Score, On-Time Delivery Rate %
CFO	Chief Financial Officer	Profitability, cost management, financial planning	Net Revenue, CAC Payback Period, Return Cost, Marketing Spend % of Revenue, Quota Attainment %
COO	Chief Operating Officer	Operational excellence, fulfilment, supply chain	On-Time Delivery Rate %, Avg Fulfillment Days, SLA Attainment, Stock-Out Rate, Operational Return Rate %
CSO	Chief Sales Officer	Sales performance, channel mix, quota management	Revenue by Channel, Quota Attainment %, Store Rankings, Category Mix, AOV

Persona	Title	Core Decision Domain	Primary KPIs
CMO	Chief Marketing Officer	Marketing ROI, acquisition, customer retention	CAC by Channel, ROAS, Paid/Organic Mix, CLV by Channel, NPS by Acquisition Channel
PM	Product Manager	Product performance, category health, inventory	Product Return Rate %, Inventory Turnover Rate, Sell-Through Rate %, Category Gross Margin %

1.3 Analytical Time Window: January 2023 – December 2025

The source Contoso data spans approximately 2005–2011. A uniform +16-year temporal shift applied at the SQL view layer maps the core transactional data to January 2023 – December 2025. This three-year window provides:

- Three complete calendar years for robust year-over-year comparison across all KPIs
- Three full fiscal cycles (July–June fiscal calendar) for budget vs actual analysis
- Sufficient depth for cohort retention analysis, customer tenure segmentation, and CLV modelling
- A period recent enough that students recognise real seasonal signals: holiday peak, Black Friday, back-to-school

Reproducibility Guarantee
The fixed reference date '2025-12-31' anchors all historical computed attributes (customer age, employee tenure, product status, store age). GETDATE() is used ONLY for three dynamic display flags in dim.vDate. This guarantees that every student running the project on any date receives identical, reproducible analytical results — a requirement for any educational analytics curriculum.

1.4 Project Coverage Summary

Status	Count	Description
✓ Fully Answerable	79	All required data, views, and DAX measures exist and are implemented or designed
⚠ Partially Answerable	4	Data exists but limited by source grain constraints or requires approximation
✗ Not Currently Answerable	6	Requires external data feeds not present in the Contoso source — gap closure plan in Document 2
Total	89	Complete business question scope across all six executive personas

2. The Three-Layer SQL Architecture

2.1 Architectural Overview

The foundational architectural decision of this project is the strict separation of the SQL database into three schema-based layers. Every other decision — from how synthetic data is generated, to how dates are shifted, to why Power Query is kept thin — flows from this single structural choice.

Architecture Data Flow
[dbo] SOURCE (Read-Only) ⟶ [gen] SYNTHETIC EXTENSION ⟶ [dim] + [fact] SEMANTIC VIEWS ⟶ Python Export ⟶ 22 Parquet Files ⟶ Power BI VertiPaq Model

Layer	Schema	Role & Philosophy	Mutability Rule
Source	[dbo]	Original ContosoRetailDW tables shipped by Microsoft. 15+ tables covering sales, inventory, products, customers, stores, employees, promotions, geography, and currencies. This layer represents the immutable ground truth of the source system.	READ-ONLY. No UPDATE, INSERT, DELETE, or ALTER ever issued. The project is fully repeatable because [dbo] is never touched.
Synthetic Extension	[gen]	Tables created to fill the six analytical gaps in the source. Extends the analytical surface without modifying or corrupting [dbo]. If the entire [gen] schema were dropped, [dbo] would be completely undamaged. This property makes the architecture reversible and auditable.	WRITABLE. Populated by Scripts 01–08. Static after Script 08 completes. All [gen] data is deterministic and reproducible on any re-run.
Semantic Views	[dim] + [fact]	22 SQL views presenting analytically-ready dimension and fact tables directly to Power BI via Parquet export. This is the semantic contract between the database and the reporting tool. ALL transformations happen here: temporal shifts, column renames, computed classifications, NULLIF guards, and pre-computed measures.	VIEW LAYER. All objects are CREATE OR ALTER VIEW — fully idempotent and safe to re-run on a live database.

2.2 The Five Core Design Principles

These five principles are the intellectual foundation of the entire project. Every SQL design choice, every naming decision, and every architectural trade-off can be traced back to one or more of these principles. Students extending or maintaining the project should internalise them before making any changes.

Principle 1: Source Immutability

No UPDATE, INSERT, DELETE, or ALTER statement is ever issued against any [dbo] table. This principle makes the project fully repeatable: dropping [gen], [dim], and [fact] and re-running all scripts from Script 00 always produces an identical result.

Industry rationale: In real enterprise BI environments, the source system is owned by a different team — the ERP team, the e-commerce platform team, or a SaaS vendor. The BI team has read-only access at best. Building the discipline of treating [dbo] as read-only mirrors this real-world constraint and prevents the most dangerous form of BI technical debt: transformation logic embedded in or destructive to the source system.

Principle 2: Schema as a Namespace Contract

Each schema is a deliberate namespace that communicates intent to any developer reading the codebase — before they read a single line of code inside the object:

- Seeing [gen.xxx] communicates: this is synthetic data generated by this project.
- Seeing [dim.vXxx] communicates: this is a clean, analytics-ready dimension view for Power BI.
- Seeing [fact.vXxx] communicates: this is a measurement fact view exposing additive metrics for Power BI.

This naming discipline eliminates ambiguity in multi-developer environments and makes the codebase self-documenting — a property that becomes critical as teams grow and projects extend over time.

Principle 3: Roche's Maxim — Push Logic Upstream

All business logic is pushed as far upstream as possible into the SQL view layer. Power Query in Power BI is kept deliberately thin — its only jobs are type enforcement (via the Schema Dictionary) and Parquet binary buffering. DAX measures are kept focused on aggregation and filter context manipulation. The database does the heavy lifting because it is architecturally better at it: set-based T-SQL operations on indexed columns outperform equivalent M or DAX transformations by orders of magnitude at 31.5 million row scale.

Concrete examples of logic pushed to the SQL layer and kept out of Power Query and DAX:

- Temporal date shifts: DATEADD(YEAR, 16, SourceDate) → computed once in SQL, never in DAX
- Age group buckets: CASE WHEN Age < 26 THEN '18–25' → computed in dim.vCustomer, available as a slicer
- Income classification: CASE WHEN YearlyIncome < 30000 THEN 'Low' → stored in dim.vCustomer
- CustomerLifecycleStage labels: New / Developing / Established / Loyal based on tenure years
- SLA tier labels: 'On Time' / 'Minor Delay' / 'Significant Delay' / 'Critical Delay' in fact.vOrderFulfillment
- StockStatus flags: 'Out of Stock' / 'Low Stock' / 'In Stock' in fact.vInventory
- GrossProfit pre-computation: SalesAmount - TotalCost → additive measure available in fact views
- NULLIF division guards: NULLIF(denominator, 0) on all SQL-layer divisions

Principle 4: Temporal Consistency — View Layer Only

The +16-year temporal shift from source dates (2007–2009) to presentation dates (2023–2025) is applied exclusively at the [dim] and [fact] view layer. All [gen] physical tables store raw source dates from the 2007–2009 era. There is a single transformation point — the view — eliminating the risk of double-shifting or inconsistent date presentation across different queries.

The fixed reference date '2025-12-31' anchors all historical computed attributes such as customer age, employee tenure, product lifecycle status, and store age. GETDATE() is reserved for only three dynamic display flags in dim.vDate (IsCurrentYear, IsCurrentMonth, IsToday) where dynamic behaviour is explicitly desired.

Principle 5: VertiPaq-First Data Types

All DateKey columns across every fact and dimension view are stored as INT in YYYYMMDD format (e.g., 20250115 for January 15, 2025). This is a conscious performance design for the VertiPaq in-memory engine that powers Power BI's data model:

- VertiPaq compresses integer columns using Value Encoding and Run Length Encoding — far more efficiently than DateTime or VARCHAR columns of equivalent cardinality.
- Star schema relationship joins that match INT on both sides (fact DateKey → dim DateKey) execute faster than any other type pairing.
- YYYYMMDD INT format allows direct date arithmetic in DAX without conversion: year = DateKey / 10000, month = (DateKey / 100) % 100.

Every date in every fact view is represented by exactly TWO columns: an INT YYYYMMDD DateKey (used as the FK for Power BI relationships) and a DATE companion column (used for human-readable display in visuals). No view exposes a date through a single column type only. This is the universal date key contract of the project.

2.3 Script Execution Order & Dependency Chain

The project is implemented across 11 SQL scripts and one Python export script that must be executed in strict dependency order. Each SQL script uses SET NOEXEC ON pre-execution checks to halt execution with an informative error if run out of sequence — protecting database integrity during development.

Order	Script File	Purpose	Depends On	Key Output
1	00_create_schemas_v2.sql	Creates [gen], [dim], [fact] schemas	None — only requires ContosoRetailDW database	3 schemas — the namespace foundation for all other objects
2	01_gen_ReferenceDimensions_v2.sql	Creates 3 small reference dimension lookup tables	Script 00 ([gen] schema must exist)	gen.DimAcquisitionChannel (7 rows), gen.DimPaymentMethod (6 rows), gen.DimReturnReason (8 rows)
3	02_gen_CustomerAcquisition_v2.sql	5-factor scoring model assigns one channel to each qualifying customer	Script 01 (gen.DimAcquisitionChannel); dbo.DimCustomer, dbo.FactOnlineSales	gen.CustomerAcquisition — one row per Person-type customer
4	03_gen_OrderPayment_v2.sql	Era-accurate probability-weighted payment method assignment per online order	Script 01 (gen.DimPaymentMethod); dbo.FactOnlineSales	gen.OrderPayment — one row per SalesOrderNumber
5	04_gen_OrderFulfillment_v2.sql	Generates complete fulfilment lifecycle for every online order	dbo.FactOnlineSales, dbo.DimCustomer, dbo.DimStore	gen.OrderFulfillment — one row per SalesOrderNumber
6	05_gen_FactMarketingSpend_v2.sql	Monthly spend by channel, internally consistent with Script 02 acquisition counts	Script 02 (actual customer counts); Script 01 (channel keys)	gen.FactMarketingSpend — one row per (month × channel), ~252 rows
7	06_gen_FactCustomerSurvey_v2.sql	Behaviourally coherent NPS/CSAT responses, correlated with real transaction history	dbo.DimCustomer, dbo.FactOnlineSales; Script 01 (channel keys)	gen.FactCustomerSurvey — one row per survey response
8	07_gen_OnlineReturnEvents_v2.sql	Event-level return records for the online channel	dbo.FactOnlineSales, dbo.DimCustomer; Script 01 (gen.DimReturnReason)	gen.OnlineReturnEvents — one row per return event

Order	Script File	Purpose	Depends On	Key Output
9	08_gen_PhysicalReturnEvents_v2.sql	Event-level return records for the physical store channel	dbo.FactSales; Script 01 (gen.DimReturnReason)	gen.PhysicalReturnEvents — one row per return event
10	09_dim_Views_v1.sql	Creates all 11 [dim] schema analytical dimension views	Scripts 00–08 must be complete; all dbo source tables verified	11 views: dim.vDate through dim.vChannel
11	10_fact_Views_v2.sql	Creates all 11 [fact] schema analytical fact views	Script 09 (all dim views must exist first); Scripts 00–08	11 views: fact.vOnlineSales through fact.vOrderPayment
12	export_contoso_views_v7.py	Reads all 22 views, exports as Snappy-compressed Parquet files	Scripts 00–10 complete; Python + ODBC environment configured	22 Parquet files + SchemaDictionary.csv in /Contoso Data Files/

3. The Source Layer: ContosoRetailDW [dbo] Inventory

3.1 Source Table Inventory

The ContosoRetailDW database ships with 15 core tables in the [dbo] schema. All are treated as immutable source records throughout this project. The table below documents each source table, its role, grain, key columns, estimated row count, and the analytical notes relevant to how each is consumed by the semantic layer.

Table	Domain	Grain	Key Column(s)	Est. Rows	Analytical Notes
FactOnlineSales	Sales	1 row per online order line item	OnlineSalesKey	~13M	Primary analytical fact. ONLY table with CustomerKey — gateway for all customer-centric analytics including CLV, cohort, and basket analysis.
FactSales	Sales	1 row per Product × Store × Date summary aggregate	SalesKey	~3.4M	Physical store sales. Summary grain — NO individual customer attribution. This structural absence is why CustomerKey is NULL by design in physical return events.
FactInventory	Inventory	1 row per Product × Store × Date daily snapshot	Composite	~5.5M	Daily inventory snapshot. Includes OnHandQuantity, OnOrderQuantity, UnitCost. Used for turnover, stock-out, and weeks-of-supply calculations.
FactSalesQuota	Planning	1 row per Employee × Date × Channel	SalesQuotaKey	~80K	Budget and quota assignments linking employees to sales targets. Backbone of CSO and CFO quota attainment reporting.
FactExchangeRate	Finance	1 row per Currency × Month (monthly grain)	ExchangeRateKey	~360	Monthly exchange rates for multi-currency revenue normalisation. Used in CFO currency exposure analysis.
DimCustomer	Customer	1 row per customer (Person AND Company types)	CustomerKey	~18K	Includes BirthDate, Geography, YearlyIncome, Education, Occupation, DateFirstPurchase. Both Person and Company types retained — see Section 3.2.
DimProduct	Product	1 row per product SKU	ProductKey	~2.5K	Snowflake structure: DimProduct → DimProductSubcategory → DimProductCategory. Flattened in dim.vProduct.
DimProductSubcategory	Product	1 row per product subcategory	ProductSubcategoryKey	~44	Middle level of 3-tier product hierarchy. Bridge between SKU and Category.
DimProductCategory	Product	1 row per product category	ProductCategoryKey	~8	Top-level product grouping: Audio, Cameras, Computers, Phones, Music/Movies, TV/Video, Games, Home Appliances.
DimStore	Store	1 row per store location	StoreKey	~307	Covers both physical stores and online stores. Links to DimGeography. Includes OpenDate, CloseDate, SellingAreaSize.
DimEmployee	HR	1 row per employee	EmployeeKey	~2K	Status column: 'Current' / 'Former'. dim.vEmployee filters to Current only for quota analytics. This scoping decision is documented in Section 4.

Table	Domain	Grain	Key Column(s)	Est. Rows	Analytical Notes
DimGeography	Geography	1 row per geographic region	GeographyKey	~700	Shared geography dimension consumed by both DimCustomer and DimStore. Column name prefixing in views (CustomerCountry, StoreCountry) prevents collision.
DimChannel	Channel	1 row per sales channel	ChannelKey	4	Online, Store, Reseller, Catalog. Required for FactSales ChannelKey FK. dim.vChannel added specifically to support fact.vStoreSales.
DimDate	Date	1 row per calendar day	Datekey	~2,922	Source range 2005–2011 (shifted +16 years to 2021–2027 in dim.vDate). All source fiscal calendar attributes are RECOMPUTED from the shifted date — not inherited from the source.
DimCurrency	Finance	1 row per currency	CurrencyKey	~10	Currency codes for multi-currency support in online sales and exchange rate facts.
DimPromotion	Marketing	1 row per promotion	PromotionKey	~80	Promotion type, discount %. PromotionKey = 1 represents 'No Promotion' — the default for all sales rows not linked to an active promotion.

3.2 Critical Source Design Decisions Affecting the Architecture

The CustomerType Decision: Including Both Person and Company

An earlier version of dim.vCustomer filtered the source to CustomerType = 'Person' only, excluding B2B company accounts. This filter was removed after it created 9.7 million orphan foreign keys — fact table rows in FactOnlineSales referencing CustomerKeys that no longer existed in the dimension. In Power BI, orphan foreign keys break referential integrity, produce incorrect DISTINCTCOUNT results, and can cause visuals to render blank or incorrect totals.

The correct approach is to include all customer types in dim.vCustomer and expose CustomerType as a slicer column, pushing filtering to the report layer where it belongs. In DAX, B2B customers return BLANK() on acquisition channel measures (because they have no row in fact.vCustomerAcquisition by design), which is the correct architectural behaviour.

The DimEmployee Current-Only Filter

dim.vEmployee filters to Status = 'Current' only. This is intentional for quota analytics — historical quota attainment is measured against the employee who held the quota at the time, and Former employees who no longer appear in quota assignments would create orphan keys in fact.vSalesQuota. The trade-off is documented: revenue-per-employee measures require accommodating this scoping in DAX (see Section 12 — Gap Closure for question S11).

The FactSales Summary Grain Limitation

dbo.FactSales records physical store sales at the Product × Store × Date summary level. There is no individual customer attribution and no order-line identity. This structural property has two downstream consequences that are architecturally correct, not bugs:

- CustomerKey is NULL by design for physical return rows in fact.vReturns — no customer-grain data exists in the physical channel source.
- SalesOrderNumber is NULL by design for physical return rows — no order-line identity exists for in-store transactions.

4. The Synthetic Extension Layer: [gen] Schema

4.1 Architecture Philosophy

The [gen] schema is the project's primary innovation. It is not a staging layer or a cache — it is a deliberate business analytics extension. Every table it contains exists to answer a specific set of executive questions that are structurally impossible to answer from the source alone. The schema was designed around three engineering mandates:

- Every [gen] table stores raw historical dates (2007–2009 era) matching the source. The +16-year shift is applied exclusively at the view layer — a single transformation point per architectural principle.
- Every [gen] fact table uses a computed BIGINT composite grain key with a UNIQUE constraint for grain enforcement. This prevents duplicate rows from re-runs without requiring a full table scan for duplicate detection.
- Every [gen] generation script embeds discoverable patterns — realistic correlations (younger customers via Social Media, high-income customers via Direct) that students can surface through their Power BI analysis, making the synthetic data pedagogically valuable.

Key Engineering Standard
All [gen] tables store raw source-era dates (2007–2009). The +16-year temporal shift to the 2023–2025 presentation era is applied EXCLUSIVELY at the [dim] and [fact] view layer. This single-transformation-point rule eliminates the risk of double-shifting and ensures [gen] tables remain the unmodified source of record for all downstream views.

4.2 Reference Dimension Tables (Script 01)

Script 01 creates three small, static reference lookup tables that form the foundational controlled vocabulary for the entire synthetic data extension. All three are manually authored — their values represent deliberate business classification choices, not computed outputs.

gen.DimAcquisitionChannel — 7 Rows

Defines the customer acquisition channels supported by the model. The IsOrganic BIT flag enables the single most critical CMO binary analysis: paid vs non-paid channel split. CAC ranges are stored as MONEY type (EstimatedCACLow, EstimatedCACHigh) — a v2.0 amendment correcting a type mismatch with the CACMidpoint computation in dim.vAcquisitionChannel.

Key	Channel Name	IsOrganic	CAC Midpoint	Design Rationale
1	Paid Search (SEM)	0 — Paid	$60	Google/Bing keyword advertising. Dominant paid channel in 2023–2025 era. High intent, moderate cost.
2	Organic Search (SEO)	1 — Organic	$11.50	Content-driven acquisition. Zero media spend but content production cost is captured in the midpoint estimate.
3	Social Media Ads	0 — Paid	$42.50	Facebook, Instagram, TikTok paid placements. Rising channel in the 2023–2025 window — YearFactor in Script 02 increases its weight year-on-year.
4	Email Marketing	1 — Organic	$8.50	Owned channel. Lowest CAC — reactivation of existing subscriber base. No media buy required.

Key	Channel Name	IsOrganic	CAC Midpoint	Design Rationale
5	Direct	1 — Organic	$0	Brand-aware visitors who arrive without any paid stimulus. MonthlySpend = $0 in FactMarketingSpend. DIVIDE() guards mandatory in DAX.
6	Referral / Affiliate	0 — Paid	$30	Word-of-mouth amplified by affiliate commission fee. Commission-based structure — no impression/click metrics.
7	Display / Retargeting	0 — Paid	$50	Banner and retargeting ads. High brand awareness, lower direct conversion rate than SEM.

gen.DimPaymentMethod — 6 Rows

Defines payment methods available in the 2023–2025 era. The IsDigital BIT flag enables digital vs traditional payment adoption analysis — a critical CFO trend metric. Methods are calibrated for contemporary realism: Digital Wallet (Apple/Google Pay) included at growing adoption rates; Cash on Delivery retained for emerging market customer segments in Asia.

Key	Payment Method	IsDigital	Notes
1	Credit Card	1 — Digital	Visa, Mastercard, Amex. Dominant global payment method across all customer segments.
2	PayPal	1 — Digital	Primary digital wallet for online transactions. Strong in European and North American markets.
3	Debit Card	1 — Digital	Direct bank debit. Common for value-conscious and younger customer segments.
4	Digital Wallet (Apple/Google Pay)	1 — Digital	Mobile-first payment. Increasing adoption share year-on-year across the 2023–2025 window.
5	Bank Transfer / Wire	0 — Traditional	Common for high-value B2B transactions. Low frequency, high value.
6	Cash on Delivery	0 — Traditional	Relevant for emerging market customers, particularly Asian geographies. Declining trend.

gen.DimReturnReason — 8 Rows

Defines the controlled vocabulary of product return reasons. Two design decisions require explanation:

First, the AppliesTo column ('Both' / 'Online Only') controls channel-appropriate reason assignment. 'Late Return (Policy Window Expired)' is designated 'Online Only' because physical in-store returns are processed at point-of-presence by staff who exercise discretion — a policy-triggered late return concept is irrelevant to that channel.

Second, the IsOperationalFailure BIT flag enables the most actionable COO analysis: controllable failures (defective product, wrong item, damaged in transit, not as described) vs customer-driven returns (changed mind, found cheaper elsewhere, duplicate order). A high operational failure rate indicates a process problem the COO can address; a high customer-preference rate indicates a pricing or expectation-setting challenge.

Key	Return Reason	AppliesTo	IsOpFailure	Business Interpretation
1	Defective / Faulty Product	Both	1 — Yes	Product quality control failure. Highest COO priority for root-cause investigation.
2	Wrong Item Received	Both	1 — Yes	Warehouse pick/pack error. Direct operational signal for fulfilment process improvement.
3	Damaged in Transit	Both	1 — Yes	Carrier or packaging failure. Connects to fulfilment carrier SLA and packaging standards.
4	Changed Mind / No Longer Needed	Both	0 — No	Customer preference return. Low priority for operational intervention.
5	Found Cheaper Elsewhere	Both	0 — No	Price competitiveness signal. Relevant for pricing strategy — not an ops failure.
6	Product Not as Described	Both	1 — Yes	Catalogue/marketing accuracy failure. Signal for PM and CMO, not just COO.
7	Duplicate Order	Both	0 — No	Customer checkout error. UX improvement opportunity — not an operations failure.
8	Late Return (Policy Window Expired)	Online Only	0 — No	Policy-triggered. Online only — physical staff discretion makes this inapplicable in-store.

4.3 Transactional Extension Tables (Scripts 02–08)

gen.CustomerAcquisition (Script 02)

Grain: one row per qualifying customer (CustomerType = 'Person', DateFirstPurchase IS NOT NULL, BirthDate IS NOT NULL). Company accounts are excluded — corporate accounts are B2B relationships, not marketing-channel acquisitions. In Power BI, B2B customers return BLANK() on acquisition channel measures, which is architecturally correct.

Channel assignment uses a five-factor multiplicative scoring model. Each customer is scored against all seven channels and assigned to the channel with the highest score:

Factor	Variable	Effect	Purpose
Base Weight	Per-channel constant	Sets prior probability of each channel being relevant	Reflects channel's general market share in retail 2023–2025
Year Factor	Changes per calendar year 2023→2025	Increases Social Media weight YoY; decreases Email weight YoY	Encodes temporal evolution of the acquisition channel mix
Age Factor	Derived from BirthDate	Younger customers score higher for Social Media; older for Direct/Referral	Demographic-channel affinity — empirically grounded pattern
Income Factor	YearlyIncome from DimCustomer	High income scores higher for Direct and Referral	High-value customers tend to be brand-aware or referred
Geo Factor	ContinentName from DimGeography	European → stronger Organic/Email; Asian → stronger Social Media	Regional digital channel maturity differences
Random Noise	RAND() seed per customer	Adds realistic variation preventing perfect determinism	Prevents artificial uniformity in the channel distribution

gen.OrderPayment (Script 03)

Grain: one row per distinct SalesOrderNumber in dbo.FactOnlineSales. Payment method assignment uses era-accurate probability weighting that evolves across the 2023–2025 window — Digital Wallet share increases year-on-year; Cash on Delivery decreases. Order value, customer geography, and customer income are secondary weighting factors: high-value orders skew toward Credit Card; Asian customers show higher Cash on Delivery rates. This produces discoverable, realistic patterns in the payment method mix.

gen.OrderFulfillment (Script 04)

Grain: one row per distinct SalesOrderNumber in dbo.FactOnlineSales. Physical store sales are intentionally excluded — they have no shipping lifecycle. The table provides the complete fulfilment lifecycle per online order:

Column Group	Columns	Business Purpose
Dates	OrderDate, ShipDate, DeliveryDate (raw source era)	Three-DateKey contract: OrderDateKey (ACTIVE), ShipDateKey (INACTIVE), DeliveryDateKey (INACTIVE) in the fact view
Timing	ProcessingDays, TransitDays, TotalFulfillmentDays	Root cause attribution: warehouse processing vs carrier transit time
Method	ShippingMethod (Standard / Express / Overnight)	Value-weighted assignment: Standard ~70%, Express ~20%, Overnight ~10%
Performance	IsOnTime (BIT), SLATier ('On Time' / 'Minor Delay' / 'Significant Delay' / 'Critical Delay')	Primary COO KPI inputs: on-time rate and delay severity distribution
Geography	GeoDistanceCategory (Domestic / Continental / International)	Geographic transit multiplier: International orders have longer transit SLA thresholds

Edge case preserved by design: ShipDate values can fall into early 2026 for orders placed in late 2025, crossing the year boundary. This is realistic — carrier transit crosses year-end. dim.vDate coverage extends to 2027-12-31 to ensure no orphan DateKey foreign keys arise from this edge case.

gen.FactMarketingSpend (Script 05)

Grain: one row per calendar month per acquisition channel (~252 rows). This is the most analytically complex generation script because spend figures must be internally consistent with the actual new-customer volumes produced by Script 02. Spend is not independently generated — it is calibrated against observed acquisition counts to produce realistic CAC values.

Key columns: MonthlySpend, Impressions, Clicks, NewCustomersAcquired (from Script 02 actuals), ComputedCAC (MonthlySpend / NewCustomers), ComputedCPC (MonthlySpend / Clicks), EstimatedRevenueDriven, ComputedROAS. The Direct channel (Key=5) always records $0 spend — DAX DIVIDE() guards are mandatory for CAC and CPC measures on this channel.

gen.FactCustomerSurvey (Script 06)

Grain: one row per survey response. Not every customer is surveyed — response rates are realistically biased: 30% for customers with 10+ orders, 20% for 2–9 orders, 15% for single-order customers. Three trigger types: Post-Purchase (all sampled customers), Quarterly (tenure > 200 days, 2+ orders), Annual (tenure > 380 days, 3+ orders).

Most importantly: survey scores are behaviourally coherent with each customer's actual transaction history in dbo.FactOnlineSales. Four built-in correlations are engineered into the score generation:

- High purchase frequency → higher NPS base score (engaged, loyal customers tend to be promoters)
- High return rate → score penalty (dissatisfaction signal from repeat return behaviour)
- Recent last purchase → recency bonus (actively engaged customers rate higher)
- High total spend → slight loyalty investment bonus (high-spend customers feel valued)

The result: students running NPS vs CLV analysis WILL find that Promoters have meaningfully higher lifetime value than Detractors. This is not coincidence — it is engineered into the generation model to produce genuine, discoverable insight in the Power BI analysis layer.

gen.OnlineReturnEvents & gen.PhysicalReturnEvents (Scripts 07–08)

These two tables provide the event-level return data entirely absent from the source. gen.OnlineReturnEvents sources from dbo.FactOnlineSales and carries: CustomerKey, ProductKey, SalesOrderNumber, SalesOrderLineNumber, ReturnReasonKey, ReturnDate, OriginalSaleDate, ReturnQuantity, ReturnAmount, RefundAmount, IsRefunded, and ReturnLagDays.

gen.PhysicalReturnEvents sources from dbo.FactSales (the summary-grain physical sales table). Because FactSales has no customer grain and no order-line grain, CustomerKey and SalesOrderNumber are structurally absent from physical return records. These columns are set to NULL by design in the unified fact.vReturns view — this is not a data quality issue; it is an honest architectural representation of the physical channel's data limitations.

5. The Temporal Shift Architecture

5.1 The Problem with 2007–2009 Source Dates

The ContosoRetailDW source data spans approximately 2005–2011. Presenting a business intelligence platform with data from 15–20 years ago to an executive audience creates immediate credibility problems: seasonal pattern references from the Great Financial Crisis era, pricing calibrated to 2008 consumer electronics costs, and trend analysis anchored to pre-smartphone market conditions all lose practical meaning. Students building modern retail analytics intuition need data that feels contemporary.

5.2 The Solution: A View-Layer-Only Temporal Shift

A uniform +16-year offset is applied to all transactional and operational dates. The critical architectural decision is the location of this transformation: exclusively at the SQL view layer ([dim] and [fact] schemas), never at the [gen] physical data layer, and never in Power Query or DAX.

Layer	Date Values Stored	Shift Applied?	Rationale
[dbo] source tables	Raw source dates (2005–2011)	No	Source immutability principle — [dbo] is never modified
[gen] synthetic tables	Raw source dates (2007–2009)	No	Single transformation point principle — shifting here would require re-running all gen scripts to change the shift amount
[dim] dimension views	Shifted dates via DATEADD(YEAR,16,...) → 2021–2027	YES	All date columns shifted except BirthDate and static reference tables
[fact] fact views	Shifted dates via DATEADD(YEAR,16,...) → 2023–2025	YES	All transactional and operational dates shifted uniformly
Power BI / DAX	Already-shifted dates received from Parquet files	No	Shift is complete before data reaches Power BI — no second transformation

Why does this matter? Applying the shift only at the view layer means there is a single transformation point. If the project required changing the shift from +16 to +17 years, only Scripts 09 and 10 would need modification — all [gen] physical tables remain unchanged. Double-shifting risk is eliminated.

5.3 The Fixed Reference Date: '2025-12-31'

All computed columns derived from dates use '2025-12-31' as the anchor point, not GETDATE(). This is the reproducibility guarantee:

Why '2025-12-31' and Not GETDATE()?
A student running this project in March 2026 and another running it in September 2028 would produce different customer age groups, different product lifecycle statuses, and different store age categories if GETDATE() were used for historical attributes. The fixed date ensures every run on any machine on any date produces bit-for-bit identical analytical outputs. This is a non-negotiable requirement for any educational or reproducible analytics project.

GETDATE() is permitted for exactly three dynamic flag columns in dim.vDate:

- IsCurrentYear: 1 if YEAR(GETDATE()) = CalendarYear — highlights the current year in visuals dynamically
- IsCurrentMonth: 1 if YEAR/MONTH(GETDATE()) matches the row — highlights the current month
- IsToday: 1 if the row's date equals CAST(GETDATE() AS DATE) — highlights today in date-level visuals

These three flags are display-only. They are never used as filters in DAX measures and do not affect any analytical KPI value.

5.4 Non-Shifted Attributes

Not every date-related attribute should be shifted. The following attributes are deliberately preserved without the +16-year offset:

Attribute	Location	Why Not Shifted
BirthDate	dim.vCustomer, dim.vEmployee	A person's date of birth is a fixed biological attribute, not a transaction event. Age is correctly computed as DATEDIFF(YEAR, BirthDate, '2025-12-31'). Shifting BirthDate would make ages appear 16 years younger.
Static reference tables	gen.DimAcquisitionChannel, gen.DimPaymentMethod, gen.DimReturnReason, DimChannel, DimCurrency	These tables carry no temporal context — they are controlled vocabulary lookups with no date columns at all. No shift is applicable.

5.5 The YYYYMMDD INT DateKey Pattern: T-SQL Implementation

Every shifted date in the view layer is represented as two columns. The INT DateKey is computed using this pattern — consistent across all 22 views:

```
-- Standard YYYYMMDD INT DateKey computation pattern (used in all fact and dim views):
CAST(
    YEAR(DATEADD(YEAR, 16, CAST(SourceDateColumn AS DATE))) * 10000
  + MONTH(DATEADD(YEAR, 16, CAST(SourceDateColumn AS DATE))) * 100
  + DAY(DATEADD(YEAR, 16, CAST(SourceDateColumn AS DATE)))
AS INT) AS DateKey,

CAST(DATEADD(YEAR, 16, CAST(SourceDateColumn AS DATE)) AS DATE) AS DisplayDate
```

The VertiPaq performance rationale for INT over DATE or DATETIME: VertiPaq's Value Encoding compresses integer columns with low cardinality (2,922 distinct dates over 8 years) to near-zero memory footprint. Star schema relationship lookups matching INT on both sides execute in the storage engine's hash-match join, which is 5–10× faster than equivalent VARCHAR or DATETIME join operations on large fact tables.

6. The Analytical View Layer: [dim] and [fact] Schemas

6.1 Cross-Cutting View Design Standards

All 22 views in the [dim] and [fact] schemas share universal design standards applied without exception. These standards are the implementation of the five principles from Section 2 at the code level.

Standard	Rule	Technical Rationale
DateKey format	INT YYYYMMDD on every date column used as a FK	VertiPaq Value Encoding: integers compress 3–10× better than DATETIME or VARCHAR. Star schema joins are fastest on matching INT types.
Dual date contract	Every date exposed as BOTH INT DateKey AND DATE companion column	INT for relationship join FK; DATE for display in visuals without format string overhead. No view exposes a date through one type only.
Division guards	NULLIF(denominator, 0) on all SQL-layer division expressions	Prevents divide-by-zero in view materialisation. Avoids silent NULL propagation that masks calculation errors.
NULL for absent keys	Absent FKs are NULL — never sentinel values like -1 or 0	Sentinel values (-1, 0) would require a matching row in the dimension table to avoid orphan key warnings in Power BI. NULL correctly renders as BLANK() in DAX — the architecturally appropriate result.
Idempotency	CREATE OR ALTER VIEW on all 22 objects	Scripts can be re-run on a live database without errors. Safe for iterative development and CI/CD pipelines.
ETL audit exclusion	ETLLoadID, LoadDate, UpdateDate excluded from all views	These are pipeline metadata columns irrelevant to analytics. Including them pollutes the Power BI Fields pane and inflates the VertiPaq model size with zero analytical value.
Snowflake flattening	Multi-level source hierarchies flattened into single dimension views	Eliminates snowflake joins in Power BI. All hierarchy levels available in one table for drill-down without intermediate bridge tables.
Geography prefixing	Geography columns prefixed with entity name (CustomerCountry, StoreCountry)	Prevents column name collision when both Customer and Store geography are in the same model. Without prefixing, Power BI cannot disambiguate two 'Country' columns.
NULL-safe naming	CONCAT_WS for all name concatenation	Standard CONCAT returns NULL if any argument is NULL. CONCAT_WS skips NULL components, producing the best available name rather than a NULL result.

6.2 Dimension Views (11 Views)

View	Source Table(s)	Grain	Key Design Decisions
dim.vDate	dbo.DimDate	1 row/day (2021–2027 shifted range)	ALL fiscal calendar attributes RECOMPUTED from the shifted date — source fiscal columns intentionally ignored to prevent stale metadata. Fiscal year starts July. IsCurrentYear/Month/Today use GETDATE() for dynamic display only.
dim.vCustomer	dbo.DimCustomer + dbo.DimGeography	1 row/customer (Person AND Company)	CustomerType filter removed after 9.7M orphan keys were detected. Computed: AgeGroup, IncomeGroup, LifeStage, CustomerLifecycleStage, Gender (normalised), MaritalStatus (normalised). Geography prefixed 'Customer'.
dim.vProduct	dbo.DimProduct + DimProductSubcatego	1 row/product SKU	Snowflake flattened to single view. UnitCost→CatalogCost, UnitPrice→CatalogPrice to signal list-price semantics.

View	Source Table(s)	Grain	Key Design Decisions
	ry + DimProductCategory		CatalogGrossMarginPct pre-computed. ProductBreadcrumb: Category > Subcategory > Product.
dim.vStore	dbo.DimStore + dbo.DimGeography	1 row/store	Geography prefixed 'Store'. StoreSizeCategory (Small/Medium/Large/Flagship) based on SellingAreaSize. StoreStatus and StoreAgeYears derived from shifted dates. Spatial types excluded.
dim.vPromotion	dbo.DimPromotion	1 row/promotion	PromotionKey=1 is 'No Promotion' — used by all sales rows without an active promotion. DAX measures must treat this key correctly to avoid inflating 'promotion-driven' revenue figures.
dim.vEmployee	dbo.DimEmployee	1 row/current employee (Status='Current' filter)	Filtered to current employees for quota analytics integrity. Former employees excluded to prevent orphan keys in fact.vSalesQuota.
dim.vPaymentMethod	gen.DimPaymentMethod	1 row/method (6 rows)	Includes IsDigital flag for digital/traditional payment split. Small static table — SINGLE_READ in export pipeline.
dim.vAcquisitionChannel	gen.DimAcquisitionChannel	1 row/channel (7 rows)	Includes IsOrganic flag and CACMidpoint for channel cost benchmarking. Small static table — SINGLE_READ in export pipeline.
dim.vReturnReason	gen.DimReturnReason	1 row/reason (8 rows)	Includes IsOperationalFailure flag and AppliesTo column for channel-appropriate filtering in return analytics.
dim.vCurrency	dbo.DimCurrency	1 row/currency (10 rows)	Provides CurrencyCode and CurrencyDescription for multi-currency display and exchange rate lookups.
dim.vChannel	dbo.DimChannel	1 row/channel (4 rows)	Added specifically to support fact.vStoreSales ChannelKey FK. Without this view, physical store channel attribution is impossible.

6.3 Fact Views (11 Views)

View	Source(s)	Grain	Key Design Decisions
fact.vOnlineSales	dbo.FactOnlineSales	1 row/online order line item	ONLY fact with CustomerKey — gateway for all CLV, cohort, repeat purchase, and basket analysis. Degenerate dimensions: SalesOrderNumber + SalesOrderLineNumber for basket analysis and TREATAS bridges. GrossProfit pre-computed in SQL. ~13M rows.
fact.vStoreSales	dbo.FactSales	1 row/Product × Store × Date summary	Summary grain — NO CustomerKey. Unique FK: ChannelKey → dim.vChannel. GrossProfit pre-computed. ~3.4M rows.
fact.vReturns	gen.OnlineReturnEvents UNION ALL gen.PhysicalReturnEvents	1 row/return event	Unified online + physical via UNION ALL. ReturnChannel discriminator. DUAL DateKey pairs: ReturnDateKey (ACTIVE) + OriginalSaleDateKey (INACTIVE for USERELATIONSHIP). CustomerKey/SalesOrderNumber NULL for physical rows by design. ReturnEventID prefixed 'ONL-' or 'PHY-' for namespace safety.
fact.vInventory	dbo.FactInventory	1 row/Product × Store × Date snapshot	Computed: InventoryValue (OnHand × UnitCost), OnOrderValue, StockStatus ('In Stock' / 'Low Stock' / 'Out of Stock'), StockCoverageRatio (NULLIF guard), AgingTier. ~5.5M rows.
fact.vSalesQuota	dbo.FactSalesQuota	1 row/Employee × Date × Channel	Provides SalesQuotaAmount for budget vs actual analysis. Links to dim.vEmployee and dim.vDate.

View	Source(s)	Grain	Key Design Decisions
fact.vExchangeRate	dbo.FactExchangeRate	1 row/Currency × Month (~360 rows)	Monthly exchange rates for multi-currency normalisation. SINGLE_READ in export pipeline.
fact.vOrderFulfillment	gen.OrderFulfillment	1 row/online order (SalesOrderNumber grain)	THREE DateKey pairs: OrderDateKey (ACTIVE), ShipDateKey (INACTIVE), DeliveryDateKey (INACTIVE). Measures: ProcessingDays, TransitDays, TotalFulfillmentDays, IsOnTime, SLATier, ShippingMethod, FulfillmentStatus.
fact.vCustomerSurvey	gen.FactCustomerSurvey	1 row/survey response	Measures: NPSScore (0–10), CSATScore (1–5), NPSCategory ('Promoter'/'Passive'/'Detractor'), NPSContribution (+1/-1/0), SurveyTrigger. SurveyResponseID is required NPS denominator key (COUNT, not SUM).
fact.vMarketingSpend	gen.FactMarketingSpend	1 row/month × channel (~252 rows)	Measures: MonthlySpend, Impressions, Clicks, NewCustomersAcquired, ComputedCAC, ComputedCPC, EstimatedRevenueDriven, ComputedROAS. Direct channel always $0. SINGLE_READ in export pipeline.
fact.vCustomerAcquisition	gen.CustomerAcquisition	1 row/Person-type customer with acquisition record	Bridge fact linking dim.vCustomer ↔ dim.vAcquisitionChannel. AcquisitionDateKey enables time-series of 'customers acquired via channel X in period Y'.
fact.vOrderPayment	gen.OrderPayment	1 row/online order (SalesOrderNumber grain)	Bridge fact linking online orders to payment methods. SalesOrderNumber is the TREATAS bridge key for joining to fact.vOnlineSales in DAX.

7. The Kimball Star Schema: Relationships & Model Design

7.1 Star Schema Architecture Principles

The Power BI data model enforces Kimball Dimensional Modelling principles strictly. All 22 tables are organised in a star schema: dimension tables store context (who, what, where, when) and fact tables store measurements (how much, how many). Every relationship is Many-to-One from fact to dimension. All filter directions are Single (dimension filters fact — never the reverse). Bi-directional filtering is disabled on all relationships — a strict Kimball requirement that prevents cross-table filter ambiguity and ensures measure isolation.

> **Why No Bi-Directional Filtering?**
> Bi-directional relationships allow a filter on one fact table to propagate through a shared dimension to another fact table, producing incorrect aggregations. For example, filtering fact.vOnlineSales by ProductKey would incorrectly filter fact.vInventory through the shared dim.vProduct relationship if bi-directional filtering were enabled. Single-direction filtering preserves measure isolation — the foundational correctness guarantee of a star schema.

7.2 Complete Relationship Registry (36 Relationships)

#	From Fact Table	From Column	To Dim Table	To Column	Card.	Active ?
1	fact.vOnlineSales	DateKey	dim.vDate	DateKey	M:1	✓ Active
2	fact.vOnlineSales	CustomerKey	dim.vCustomer	CustomerKey	M:1	✓ Active
3	fact.vOnlineSales	ProductKey	dim.vProduct	ProductKey	M:1	✓ Active
4	fact.vOnlineSales	StoreKey	dim.vStore	StoreKey	M:1	✓ Active
5	fact.vOnlineSales	PromotionKey	dim.vPromotion	PromotionKey	M:1	✓ Active
6	fact.vOnlineSales	CurrencyKey	dim.vCurrency	CurrencyKey	M:1	✓ Active
7	fact.vStoreSales	DateKey	dim.vDate	DateKey	M:1	✓ Active
8	fact.vStoreSales	ProductKey	dim.vProduct	ProductKey	M:1	✓ Active
9	fact.vStoreSales	StoreKey	dim.vStore	StoreKey	M:1	✓ Active
10	fact.vStoreSales	PromotionKey	dim.vPromotion	PromotionKey	M:1	✓ Active
11	fact.vStoreSales	CurrencyKey	dim.vCurrency	CurrencyKey	M:1	✓ Active

#	From Fact Table	From Column	To Dim Table	To Column	Card.	Active ?
12	fact.vStoreSales	ChannelKey	dim.vChannel	ChannelKey	M:1	✓ Active
13	fact.vReturns	ReturnDateKey	dim.vDate	DateKey	M:1	✓ Active
14	fact.vReturns	OriginalSaleDateKey	dim.vDate	DateKey	M:1	✗ INACTIVE
15	fact.vReturns	CustomerKey	dim.vCustomer	CustomerKey	M:1	✓ Active
16	fact.vReturns	ProductKey	dim.vProduct	ProductKey	M:1	✓ Active
17	fact.vReturns	StoreKey	dim.vStore	StoreKey	M:1	✓ Active
18	fact.vReturns	ReturnReasonKey	dim.vReturnReason	ReturnReasonKey	M:1	✓ Active
19	fact.vInventory	DateKey	dim.vDate	DateKey	M:1	✓ Active
20	fact.vInventory	ProductKey	dim.vProduct	ProductKey	M:1	✓ Active
21	fact.vInventory	StoreKey	dim.vStore	StoreKey	M:1	✓ Active
22	fact.vSalesQuota	DateKey	dim.vDate	DateKey	M:1	✓ Active
23	fact.vSalesQuota	EmployeeKey	dim.vEmployee	EmployeeKey	M:1	✓ Active
24	fact.vExchangeRate	DateKey	dim.vDate	DateKey	M:1	✓ Active
25	fact.vExchangeRate	CurrencyKey	dim.vCurrency	CurrencyKey	M:1	✓ Active
26	fact.vOrderFulfillment	OrderDateKey	dim.vDate	DateKey	M:1	✓ Active
27	fact.vOrderFulfillment	ShipDateKey	dim.vDate	DateKey	M:1	✗ INACTIVE
28	fact.vOrderFulfillment	DeliveryDateKey	dim.vDate	DateKey	M:1	✗ INACTIVE
29	fact.vCustomerSurvey	SurveyDateKey	dim.vDate	DateKey	M:1	✓ Active
30	fact.vCustomerSurvey	CustomerKey	dim.vCustomer	CustomerKey	M:1	✓ Active
31	fact.vMarketingSpend	MonthStartDateKey	dim.vDate	DateKey	M:1	✓ Active
32	fact.vMarketingSpend	AcquisitionChannelKey	dim.vAcquisitionChannel	AcquisitionChannelKey	M:1	✓ Active
33	fact.vCustomerAcquisition	CustomerKey	dim.vCustomer	CustomerKey	M:1	✓ Active

#	From Fact Table	From Column	To Dim Table	To Column	Card.	Active ?
34	fact.vCustomerAcquisition	AcquisitionChannelKey	dim.vAcquisitionChannel	AcquisitionChannelKey	M:1	✓ Active
35	fact.vCustomerAcquisition	AcquisitionDateKey	dim.vDate	DateKey	M:1	✓ Active
36	fact.vOrderPayment	PaymentMethodKey	dim.vPaymentMethod	PaymentMethodKey	M:1	✓ Active

7.3 Role-Playing Dimensions

dim.vDate serves as a role-playing dimension in three fact views — a single physical table acting in multiple semantic roles simultaneously. This is a core Kimball pattern. In Power BI, a single physical table can have multiple relationships to the same target: exactly one can be ACTIVE (used by default); all others must be INACTIVE and activated on-demand in DAX via USERELATIONSHIP().

Fact View	Active DateKey	Inactive DateKey(s)	DAX USERELATIONSHIP Pattern	Business Question Enabled
fact.vReturns	ReturnDateKey	OriginalSaleDateKey	CALCULATE([Return Count], USERELATIONSHIP(fact_vReturns[OriginalSaleDateKey], dim_vDate[DateKey]))	Returns from sales cohort in period X — return lag analysis by original sale date
fact.vOrderFulfillment	OrderDateKey	ShipDateKey, DeliveryDateKey	CALCULATE([Orders Shipped], USERELATIONSHIP(fact_vOrderFulfillment[ShipDateKey], dim_vDate[DateKey]))	Orders shipped in period X; orders delivered in period X — separate from order placement date

7.4 Cross-Fact Bridge Pattern: TREATAS in DAX

Two fact views — fact.vOrderFulfillment and fact.vOrderPayment — connect to fact.vOnlineSales via shared degenerate dimension keys (SalesOrderNumber) rather than formal Power BI model relationships. A formal relationship between two fact tables creates ambiguous filter propagation paths — a model anti-pattern in Kimball methodology.

The correct pattern is TREATAS in DAX, which treats a column from one table as if it were a column from another for filtering purposes, without creating a model relationship:

```
-- Example: Fulfilment metrics filtered to orders matching a customer segment
Avg Fulfillment Days for Segment =
CALCULATE(
    [Avg Total Fulfillment Days],
    TREATAS(
        VALUES(fact_vOnlineSales[SalesOrderNumber]),
        fact_vOrderFulfillment[SalesOrderNumber]
    )
)
```

This enables filtering fulfilment performance, payment method distribution, and return analysis to any subset of online orders defined by customer segment, product category, promotion, or time period — without adding fact-to-fact model relationships that would corrupt the star schema.

8. The Python Export Pipeline

8.1 Pipeline Purpose & Technology Choices

The Python export pipeline (export_contoso_views_v7.py) bridges the SQL Server semantic layer and the Power BI model. It reads all 22 views from SQL Server and serialises them as Snappy-compressed Parquet files stored on SharePoint for consumption by Power BI via cloud-based file references.

Parquet is chosen over CSV, JSON, or Excel for three technical reasons: its columnar storage format allows Power Query to read only the columns it needs (column pruning); Snappy compression reduces file sizes 5–10× vs CSV while preserving full type fidelity; and Parquet's embedded schema metadata (column names and types) integrates with the Schema Dictionary defensive re-typing pattern.

8.2 Adaptive Export Strategy: SINGLE_READ vs CHUNKED_STREAM

Version 7 of the export script classifies every view into one of two export strategies based on estimated row count:

Strategy	Criteria	Mechanism	Performance Gain
SINGLE_READ	View row count < 200,000 rows	pd.read_sql() with no chunksize parameter — one DataFrame → one Arrow Table → one writer.write_table() call → writer.close()	Eliminates per-chunk overhead (schema inference, DataFrame construction, Arrow conversion, compression write) for all 13 small views. Removing chunksize from a 7-row table eliminates ~12 unnecessary operations.
CHUNKED_STREAM	View row count ≥ 200,000 rows	stream_results=True + chunksize=200,000 row iteration — streams rows in pages to avoid RAM exhaustion on 8 GB developer machines	Allows a 13M-row fact table to export without requiring 13M rows in memory simultaneously. Trade-off: per-chunk overhead is acceptable because the alternative is OOM crash.

Export Strategy	Views Using This Strategy
SINGLE_READ (13 views)	All 11 dim views + fact.vExchangeRate + fact.vMarketingSpend
CHUNKED_STREAM (9 views)	fact.vOnlineSales (~13M), fact.vStoreSales (~3.4M), fact.vInventory (~5.5M), fact.vReturns, fact.vOrderFulfillment, fact.vCustomerSurvey, fact.vSalesQuota, fact.vCustomerAcquisition, fact.vOrderPayment

8.3 Performance Optimisations (PERF 1–8)

Version 7 incorporates eight targeted performance optimisations over the previous version:

Optimisation	Technique Applied	Technical Rationale
PERF 1	Adaptive single-read vs chunked-stream classification	Eliminates per-chunk overhead for 13 small views that do not need streaming

Optimisation	Technique Applied	Technical Rationale
PERF 2	WITH (NOLOCK) on all SELECT queries	Bypasses SQL Server's shared lock acquisition (S-locks on each 8KB data page). For fact.vOnlineSales (~100K pages), this removes ~100K lock acquisitions per export. Acceptable because [dbo] and [gen] are static during export — dirty reads are impossible on static tables.
PERF 3	pool_pre_ping=False in SQLAlchemy engine	Removes one SELECT 1 health-check round-trip per connection checkout — 22 saved round-trips over the full export run.
PERF 4	use_setinputsizes=False in connect_args	Disables cursor.setinputsizes() call in PyODBC, which negotiates parameter metadata with SQL Server even for fully parameter-free SELECT * queries. Removes one server round-trip per query.
PERF 5	HAS_DATETIME_COLS computed once per view, reused per chunk	Eliminates repeated select_dtypes() column scanning across all chunks for views with no datetime columns — e.g., dim.vPaymentMethod (7 rows, 0 datetime columns) would have scanned all columns on every chunk if chunked.
PERF 6	ProcessPoolExecutor replacing ThreadPoolExecutor	Bypasses Python's Global Interpreter Lock (GIL). PyArrow Parquet compression is CPU-bound; with ThreadPoolExecutor all workers compete for one CPU core. ProcessPoolExecutor distributes work across physical CPU cores, achieving true parallelism.
PERF 7	SQLAlchemy engine instantiated inside each worker process	SQLAlchemy engines cannot be safely shared across process boundaries. Moving engine creation into the worker function gives each process its own independent connection pool — prevents connection corruption in multiprocessing.
PERF 8	yield_per=CHUNK_SIZE on PyODBC cursor via SQLAlchemy execution_options	Instructs the underlying PyODBC cursor to pre-fetch rows in bulk (arraysize) matching the chunk size rather than fetching row-by-row. Dramatically reduces the number of network round-trips between SQL Server and the Python client for large result sets.

8.4 Schema Dictionary: Defensive Re-Typing in Power Query

The export pipeline generates a SchemaDictionary.csv alongside the 22 Parquet files. This CSV maps every (SchemaName, ViewName, ColumnName) triple to its canonical Power Query M type string:

SQL Data Type	M Type String	Notes
int, bigint, smallint, tinyint	Int64.Type	All integer variants map to Int64 — Power Query's universal integer type
varchar, nvarchar, char, nchar	type text	All text variants map to type text
date	type date	DATE columns remain date type — not promoted to datetime
datetime, datetime2	type datetime	DATETIME columns map to datetime
money, smallmoney	Currency.Type	Currency.Type preserves 4-decimal precision and triggers currency formatting in Power BI
decimal, numeric	type number	General numeric type for non-monetary decimals
bit	type logical	BIT → Boolean logical — enables direct use as filter without conversion

In Power Query, the Schema Dictionary is loaded first and applied as a Table.TransformColumnTypes() call to every imported Parquet table. This defensive re-typing ensures that Parquet's own embedded type metadata is overridden by the authoritative SQL-layer type definitions. Without this pattern, Power Query occasionally misinterprets Parquet INT32 as Int64.Type when it should remain Currency.Type, silently corrupting money column formatting.

8.5 Parquet File Inventory (22 Files + Schema Dictionary)

Parquet File	Source View	Est. Rows	Export Strategy
dim_vDate.parquet	dim.vDate	~2,922	Single Read
dim_vCustomer.parquet	dim.vCustomer	~18,000	Single Read
dim_vProduct.parquet	dim.vProduct	~2,500	Single Read
dim_vStore.parquet	dim.vStore	~307	Single Read
dim_vPromotion.parquet	dim.vPromotion	~80	Single Read
dim_vEmployee.parquet	dim.vEmployee	~2,000	Single Read
dim_vPaymentMethod.parquet	dim.vPaymentMethod	6	Single Read
dim_vAcquisitionChannel.parquet	dim.vAcquisitionChannel	7	Single Read
dim_vCurrency.parquet	dim.vCurrency	~10	Single Read
dim_vReturnReason.parquet	dim.vReturnReason	8	Single Read
dim_vChannel.parquet	dim.vChannel	4	Single Read
fact_vOnlineSales.parquet	fact.vOnlineSales	~13,000,000	Chunked Stream
fact_vStoreSales.parquet	fact.vStoreSales	~3,400,000	Chunked Stream
fact_vReturns.parquet	fact.vReturns	~500,000	Chunked Stream
fact_vInventory.parquet	fact.vInventory	~5,500,000	Chunked Stream
fact_vSalesQuota.parquet	fact.vSalesQuota	~80,000	Chunked Stream
fact_vExchangeRate.parquet	fact.vExchangeRate	~360	Single Read
fact_vOrderFulfillment.parquet	fact.vOrderFulfillment	~300,000	Chunked Stream
fact_vCustomerSurvey.parquet	fact.vCustomerSurvey	~50,000	Chunked Stream
fact_vMarketingSpend.parquet	fact.vMarketingSpend	~252	Single Read
fact_vCustomerAcquisition.parquet	fact.vCustomerAcquisition	~18,000	Chunked Stream
fact_vOrderPayment.parquet	fact.vOrderPayment	~300,000	Chunked Stream
SchemaDictionary.csv	INFORMATION_SCHEMA.COLUMNS query	~350 column mappings	N/A — generated separately

9. Database Architecture Summary

9.1 The Full Data Flow in One View

The complete journey from Microsoft's sample database to an analytically-ready Power BI model is achieved in six steps, each governed by the design principles established in Section 2:

1. Script 00 creates the [gen], [dim], and [fact] schema namespaces — the three-layer contract.
2. Scripts 01–08 populate 10 [gen] tables with synthetic data that closes the six analytical gaps in [dbo], producing discoverable, realistic patterns calibrated to the 2023–2025 business era.
3. Script 09 creates 11 [dim] views that flatten, rename, classify, and temporally shift source and synthetic dimension data into clean Power BI-ready dimension tables.
4. Script 10 creates 11 [fact] views that apply the +16-year temporal shift, pre-compute GrossProfit and stock metrics, enforce dual DateKey contracts, and expose clean measurement surfaces.
5. export_contoso_views_v7.py reads all 22 views using adaptive export strategies and 8 performance optimisations, serialising them to Snappy-compressed Parquet files with a companion Schema Dictionary.
6. Power BI loads the 22 Parquet files via SharePoint, applies defensive re-typing from the Schema Dictionary, establishes 36 star schema relationships, and delivers the VertiPaq-compressed model to the reporting layer.

9.2 What Document 2 Covers

Document 2 of this series covers the Power BI layer in full: the Two-File Architecture, Power Query M patterns, the VertiPaq data model configuration, all seven DAX analytical pipelines with their 49 measures, the complete business questions coverage matrix (89 questions across six personas), the gap closure strategy for achieving 100% coverage, the dashboard design for all six executive personas using the Z-Pattern visual hierarchy, and the improvement plan for the full project.

Document 3 covers the technical reference appendices: full script execution order, column dictionary, complete relationship map, and the full DAX formula library for all 49 measures.